

**UNIVERSIDADE FEEVALE**

**CURSOS DE CIÊNCIA DA COMPUTAÇÃO**

**E SISTEMAS DE INFORMAÇÃO**

**SIMULADOR DE ESCALONAMENTO DE PROCESSOS**

**Round Robin com Feedback**

**Relatório Técnico**

**Trabalho Final**

Disciplina: Sistemas Operacionais

Cauê Cavalheiro Schilling

Igor Terres de Oliveira

João Miguel Vieira Dalsoto

Josué Henrique Becker Schwartzhaupt

Lucas dos Passos Almeida

Victor Matheus Herrmann

Novo Hamburgo

RS

2026

## SUMÁRIO

|   |                             |    |
|---|-----------------------------|----|
| 1 | INTRODUÇÃO.....             | 3  |
| 2 | DESCRIÇÃO DO ALGORITMO..... | 5  |
| 3 | PREMISSAS DO SIMULADOR..... | 7  |
| 4 | ESTRUTURAS DE DADOS.....    | 9  |
| 5 | FLUXO DE EXECUÇÃO.....      | 11 |
| 6 | EXECUÇÃO DO PROJETO.....    | 14 |
| 7 | SAÍDA DO SIMULADOR.....     | 17 |
| 8 | CONCLUSÃO.....              | 21 |
|   | REFERÊNCIAS.....            | 23 |

# 1 INTRODUÇÃO

O escalonamento de processos é um dos mecanismos fundamentais de qualquer sistema operacional moderno. Sua função é determinar qual processo ocupa a Unidade Central de Processamento (CPU) em cada instante, de modo a maximizar a utilização dos recursos, reduzir o tempo de resposta e garantir justiça entre os processos concorrentes. Sem um escalonador eficiente, sistemas multiprogramados seriam inviáveis, pois múltiplos processos disputariam continuamente o acesso à CPU sem critério.

O presente trabalho tem como objetivo documentar e analisar o simulador de escalonamento de processos desenvolvido pelo grupo como trabalho final da disciplina de Sistemas Operacionais do Curso de Ciência da Computação e Sistemas de Informação da Universidade Feevale. O simulador implementa o algoritmo Round Robin com Feedback (também conhecido como Multilevel Feedback Queue, MFQ) na linguagem Java, explorando conceitos fundamentais de gerenciamento de processos, incluindo preempção, bloqueio por operações de entrada e saída (I/O), retorno de I/O e cálculo de métricas de desempenho como turnaround e tempo de espera.

A escolha do algoritmo Round Robin com Feedback se justifica pela sua ampla utilização em sistemas operacionais reais, como o Unix e seus derivados, por combinar equidade na distribuição de tempo de CPU com sensibilidade ao comportamento dos processos: processos que consomem muito CPU e não solicitam I/O são gradualmente penalizados, enquanto processos que realizam operações de I/O com frequência recebem prioridade elevada ao retornar do bloqueio.

Este relatório está organizado em oito seções. A Seção 2 descreve o algoritmo implementado. A Seção 3 apresenta as premissas adotadas. A Seção 4 detalha as estruturas de dados. A Seção 5 explica o fluxo de execução. A Seção 6 instrui como compilar e executar o projeto. A Seção 7 exhibe exemplos de saída comentados. Por fim, a Seção 8 apresenta as conclusões e possíveis melhorias.

## 2 DESCRIÇÃO DO ALGORITMO

O algoritmo Round Robin (RR) é um escalonador preemptivo baseado no conceito de fatia de tempo (time slice), denominada quantum. A cada quantum, o processo em execução é interrompido e retorna ao final da fila de prontos, dando oportunidade a outro processo. O RR puro trata todos os processos com a mesma prioridade, o que pode ser ineficiente quando há processos com comportamentos muito distintos.

O Round Robin com Feedback aprimora o RR clássico ao introduzir múltiplas filas de prioridade. No modelo implementado, existem duas filas de prontos: a fila de alta prioridade e a fila de baixa prioridade. Os processos ingressam sempre pela fila de alta prioridade. Quando um processo consome todo o seu quantum sem finalizar e sem solicitar I/O, é considerado CPU-bound e sofre uma penalização, sendo movido para a fila de baixa prioridade. Por outro lado, processos que solicitam I/O antes de esgotar o quantum demonstram comportamento I/O-bound e, ao retornarem do bloqueio, podem voltar à fila de alta prioridade (no caso de FITA ou IMPRESSORA) ou à fila de baixa prioridade (no caso de DISCO).

Essa diferenciação se baseia no princípio de que processos I/O-bound possuem baixa demanda por CPU e alta necessidade de responsividade, enquanto processos CPU-bound precisam de grandes fatias de processamento. Ao priorizar os primeiros, o sistema mantém boa interatividade sem prejudicar o throughput geral.

O escalonador implementado no projeto opera de acordo com as seguintes regras:

- Processos novos são admitidos na fila de alta prioridade conforme seu tempo de chegada.
- A CPU sempre prefere a fila de alta prioridade. Apenas quando esta está vazia, processa a fila de baixa.

- Se ambas as filas estiverem vazias, a CPU registra ociosidade e avança o tempo em uma unidade.
- Ao finalizar o quantum, processos não concluídos e sem I/O pendente vão para a fila de baixa (preempção).
- Processos que solicitam I/O saem da CPU, vão para a fila de I/O e aguardam o tempo de serviço do dispositivo.
- Ao concluir o I/O, retornam à fila de alta (FITA ou IMPRESSORA) ou baixa (DISCO).
- Processos cujo tempo total de CPU é atingido são marcados como finalizados e removidos do sistema.

Essa política equilibra os objetivos de maximizar o uso da CPU, minimizar o tempo de resposta para processos interativos e garantir que processos de longa duração também obtenham acesso ao processador.

### 3 PREMISSAS DO SIMULADOR

Esta seção descreve os parâmetros e convenções adotados na implementação do simulador, conforme definidos no código-fonte do projeto.

#### 3.1 Quantum

O quantum adotado é de 3 unidades de tempo (definido pela constante `QUANTUM = 3`). Isso significa que cada processo pode ocupar a CPU por no máximo 3 unidades antes de ser preemptado.

#### 3.2 Número máximo de processos

O simulador suporta até 8 processos simultâneos (`MAX_PROC = 8`). Todos os arrays do PCB são pré-alocados com esse tamanho.

#### 3.3 Processos simulados

Na versão atual, quatro processos são criados estaticamente no início da simulação pela função `inicializarProcessos()`, conforme o Quadro 1:

*Quadro 1 - Processos configurados na simulação*

| PID | Chegada (t) | Tempo de CPU | Tipo de I/O | Início do I/O |
|-----|-------------|--------------|-------------|---------------|
| P0  | 0           | 4 u.t.       | DISCO       | 2 u.t.        |
| P1  | 4           | 5 u.t.       | Nenhum      | -             |
| P2  | 8           | 7 u.t.       | Nenhum      | -             |
| P3  | 12          | 8 u.t.       | Nenhum      | -             |

A geração aleatória de processos foi implementada durante o desenvolvimento do simulador, incluindo a utilização de uma semente fixa para garantir a reprodutibilidade dos testes. No entanto, para a validação da lógica do escalonador e a análise de cenários específicos, optou-se por utilizar processos definidos manualmente na versão apresentada. Essa abordagem permitiu reproduzir casos particulares de forma controlada, facilitando a verificação de comportamentos como preempções, mudanças de fila, operações de I/O e finalização de processos. A funcionalidade de geração aleatória permanece disponível no código-fonte, estando apenas com sua inicialização comentada, podendo ser habilitada facilmente para a execução de simulações com cargas de trabalho geradas automaticamente.

### 3.4 Tempos de I/O

Os tempos de serviço dos dispositivos de I/O são fixos e definidos pela função `duracaoIo()`, conforme apresentado no Quadro 2:

*Quadro 2 - Duração dos dispositivos de I/O*

| Dispositivo | Duração (unidades de tempo) |
|-------------|-----------------------------|
| DISCO       | 8 u.t.                      |
| FITA        | 3 u.t.                      |
| IMPRESSORA  | 5 u.t.                      |



## 4 ESTRUTURAS DE DADOS

O simulador representa o Process Control Block (PCB) por meio de arrays paralelos, em vez de uma classe ou struct dedicada. Cada índice dos arrays corresponde ao mesmo processo. Essa abordagem é simples e direta, adequada para um número pequeno e fixo de processos.

### 4.1 Bloco de Controle de Processo (PCB)

O PCB armazena todas as informações necessárias para controlar o ciclo de vida de cada processo. O Quadro 3 descreve os campos utilizados:

*Quadro 3 - Campos do PCB implementados como arrays paralelos*

| Campo (array)       | Descrição                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------|
| pid[]               | Identificador único do processo (PID), equivale ao próprio índice.                                  |
| ppid[]              | PID do processo pai. Fixado em -1 (sem pai) nesta implementação.                                    |
| tempoChegada[]      | Instante de tempo em que o processo entra no sistema.                                               |
| tempoTotal[]        | Tempo total de CPU necessário para o processo concluir.                                             |
| tempoProcessado[]   | Tempo de CPU já consumido pelo processo até o momento atual.                                        |
| status[]            | Estado atual do processo: NOVO(0), PRONTO(1), EXECUTANDO(2), BLOQUEADO(3) ou FINALIZADO(4).         |
| tipoIO[]            | Tipo do dispositivo de I/O que o processo utilizará: NENHUM(0), DISCO(1), FITA(2) ou IMPRESSORA(3). |
| inicioProximoIo[]   | Instante de CPU (relativo ao processo) em que ocorrerá a solicitação de I/O.                        |
| tempoIoProcessado[] | Quanto do tempo de I/O já foi cumprido pelo processo no dispositivo.                                |
| instanteEntradaIo[] | Instante de simulação em que o processo entrou na fila de I/O.                                      |

## 4.2 Filas de Escalonamento

O simulador utiliza três filas implementadas como `Queue<Integer>` com `LinkedList` como estrutura subjacente, armazenando os PIDs dos processos:

- `filaAlta`: fila de alta prioridade. Processos chegam aqui inicialmente e retornam após I/O por FITA ou IMPRESSORA.
- `filaBaixa`: fila de baixa prioridade. Processos preemptados ou retornados do I/O de DISCO são inseridos aqui.
- `filaIO`: fila de espera por I/O. Processos bloqueados aguardam nesta fila enquanto o dispositivo realiza o serviço.

## 4.3 Métricas e Linha do Tempo

Além do PCB e das filas, o simulador mantém estruturas auxiliares para coleta de métricas:

- `contPreempcoes[]`: contador de quantas vezes cada processo foi preemptado.
- `instanteFinalizacao[]`: instante em que cada processo foi finalizado.
- `tempoDeEspera[]`: tempo total que cada processo passou aguardando na fila de prontos.
- `cpuOciosaTotal`: contador de unidades de tempo em que a CPU ficou sem processos para executar.
- `linhaDoTempo`: `Map<Integer, List<String>>` que associa cada instante de tempo a uma lista de eventos ocorridos, usada para imprimir o log cronológico da simulação.

## **5 FLUXO DE EXECUÇÃO**

Esta seção descreve detalhadamente o ciclo de vida dos processos no simulador, desde a criação até a finalização, percorrendo cada transição de estado possível.

### **5.1 Criação e Admissão**

Todos os processos são registrados no início da simulação pela função `inicializarProcessos()`. Cada processo é inserido com status NOVO. O loop principal de simulação, a cada iteração, verifica quais processos novos possuem `tempoChegada` menor ou igual ao tempo corrente, transita seu status para PRONTO e os insere na `filaAlta` pela função `moverNovosProcessosParaFilaAlta()`. Processos com o mesmo tempo de chegada são ordenados pelo `tempoChegada` antes de ser adicionados à fila, garantindo consistência.

### **5.2 Seleção e Execução**

A CPU seleciona o próximo processo a executar seguindo a política de prioridade: primeiro verifica `filaAlta`; se vazia, verifica `filaBaixa`; se ambas vazias, incrementa o contador de CPU ociosa e avança o tempo. O processo selecionado tem seu status alterado para EXECUTANDO e é executado pela função `executarPorAteUmQuantum()`. Esta função calcula o tempo real de execução como o mínimo entre: o tempo restante do processo (`tempoTotal - tempoProcessado`), o tempo até o próximo I/O (se houver) e o quantum (3 unidades).

### **5.3 Preempção**

Ao término do quantum, se o processo não foi finalizado e não solicitou I/O, ocorre a preempção. O processo tem seu status alterado para PRONTO, é inserido na `filaBaixa` e seu

contador de preempções é incrementado. O evento é registrado na linha do tempo. Esse mecanismo implementa a política de feedback: processos que consomem o quantum completo são rebaixados para a fila de menor prioridade.

#### **5.4 Bloqueio por I/O**

Um processo solicita I/O quando seu `tempoProcessado` atinge o valor definido em `inicioProximoIo`. Nesse momento, a função `processoSolicitouIO()` retorna verdadeiro, o status é alterado para BLOQUEADO, o instante de entrada na fila de I/O é registrado e o processo é inserido na `filaIO`. O tipo do dispositivo (DISCO, FITA ou IMPRESSORA) e a duração correspondente determinam quanto tempo o processo ficará bloqueado.

A função `atualizarFilasDeIo()` é chamada a cada iteração do loop principal, processando o tempo de I/O de forma sequencial: cada processo na fila de I/O avança seu `tempoIoProcessado` proporcionalmente ao tempo decorrido desde a última verificação (variável `tempoAnterior`).

#### **5.5 Retorno de I/O**

Quando o I/O de um processo é concluído (`tempoIoProcessado >= duracaoIo(tipoIO)`), a função `finalizarIo()` é chamada. O processo é removido da `filaIO`, seu status é alterado para PRONTO e é recolocado em uma das filas de prontos conforme a política de retorno:

- DISCO: processo retorna à `filaBaixa` (penalizado por ser um dispositivo lento e de uso mais intenso).
- FITA ou IMPRESSORA: processo retorna à `filaAlta` (beneficiado por ser um dispositivo de uso mais leve).

Os campos `tempoIoProcessado` e `tipoIO` são zerados após o retorno, preparando o processo para uma eventual futura solicitação de I/O.

## **5.6 Finalização**

O processo é finalizado quando `tempoProcessado == tempoTotal`, ou seja, quando todo o tempo de CPU necessário foi cumprido. O status é alterado para `FINALIZADO`, o instante de finalização é registrado e o evento é logado na linha do tempo. O loop principal continua até que todos os processos estejam com status `FINALIZADO`, verificado pela função `existemProcessosNaoFinalizados()`.

## 6 EXECUÇÃO DO PROJETO

Esta seção descreve como compilar e executar o simulador. O projeto é desenvolvido em Java 21 e utiliza Apache Maven como ferramenta de gerenciamento de dependências e build.

### 6.1 Pré-requisitos

- Java Development Kit (JDK) 21 ou superior.
- Apache Maven 3.8 ou superior (opcional, para uso com o pom.xml).
- Git, para clonar o repositório.

### 6.2 Clonando o Repositório

Para obter o código-fonte, execute o comando a seguir em um terminal:

```
git clone
https://github.com/JosueSchwartzhaupt/Escalonador-De-Processos-SisOp.git
cd Escalonador-De-Processos-SisOp
```

### 6.3 Compilação e Execução via Linha de Comando

Para compilar e executar o simulador diretamente com javac e java, a partir da raiz do projeto:

```
# Compilar
javac -d out src/main/java/com/sisop/Main.java

# Executar
java -cp out com.sisop.Main
```

O README do repositório também apresenta uma forma simplificada de execução, considerando que o arquivo principal esteja no diretório corrente:

```
javac Main.java
java Main
```

## 6.4 Compilação e Execução via Maven

O projeto inclui um arquivo pom.xml configurado para Java 21. Para compilar e executar com Maven:

```
# Compilar e empacotar
mvn compile

# Executar via Maven
mvn exec:java -Dexec.mainClass="com.sisop.Main"
```

## 6.5 Docker (Bônus)

Como funcionalidade adicional, foi disponibilizada uma configuração Docker para facilitar a execução do simulador em diferentes ambientes. O uso de contêineres garante que a aplicação seja executada de forma padronizada, sem a necessidade de instalar manualmente Java ou Maven na máquina do usuário.

O projeto utiliza Maven para compilar o código-fonte e gerar o arquivo executável JAR. No Dockerfile foi adotada a estratégia de multi-stage build. No primeiro estágio, utilizando a imagem maven:3.9.11-eclipse-temurin-21, o projeto é compilado por meio do comando:

**mvn clean package -DskipTests**

No segundo estágio, uma imagem mais leve (eclipse-temurin:21-jre) é utilizada apenas para executar a aplicação. O arquivo JAR gerado é copiado para a imagem final e iniciado automaticamente pelo comando: `java -jar app.jar`

Para construir e executar o simulador utilizando Docker, devem ser utilizados os seguintes comandos:

**docker build -t so-escalador-grupo-4 .**

**docker run --rm so-escalador-grupo-4**

A opção `--rm` remove automaticamente o contêiner após a execução, mantendo o ambiente limpo.



## 7 SAÍDA DO SIMULADOR

Ao ser executado com os processos configurados em `inicializarProcessos()`, o simulador produz uma saída estruturada em três blocos: configurações iniciais, linha do tempo de eventos e resumo final. As subseções a seguir descrevem cada bloco e interpretam os eventos registrados.

### 7.1 Cabeçalho de Configuração

A primeira saída exibida pelo método `imprimirConfiguracoes()` apresenta os parâmetros do simulador:

```
=====
Simulador Round Robin com Feedback
quantum=3 | max_processos=8
I/O: DISCO=8u (->BAIXA) | FITA=3u (->ALTA) | IMPR=5u (->ALTA)
=====
```

Essa saída confirma os valores de quantum (3 u.t.), capacidade máxima de processos (8) e as políticas de retorno de I/O para cada dispositivo.

### 7.2 Linha do Tempo

A linha do tempo lista todos os eventos relevantes em ordem cronológica, identificando o instante de simulação. A seguir, apresenta-se a saída esperada com base nos quatro processos configurados:

```

=====
LINHA DO TEMPO
=====

[t=000] P0 criado → fila ALTA
        CPU executa P0 por 2 unidades
[t=002] P0 solicitou I/O DISCO
        CPU ociosa
[t=003] CPU ociosa
[t=004] P1 criado → fila ALTA
        CPU executa P1 por 3 unidades
[t=007] P1 sofreu preempção -> fila BAIXA
        CPU executa P1 por 2 unidades
[t=008] P2 criado → fila ALTA
[t=009] P1 finalizado
        CPU executa P2 por 3 unidades
[t=010] P0 retornou do DISCO -> fila BAIXA
[t=012] P2 sofreu preempção -> fila BAIXA
        P3 criado → fila ALTA
        CPU executa P3 por 3 unidades
[t=015] P3 sofreu preempção -> fila BAIXA
        CPU executa P2 por 3 unidades
[t=018] P2 sofreu preempção -> fila BAIXA
        CPU executa P0 por 2 unidades
[t=020] P0 finalizado
        CPU executa P3 por 3 unidades
[t=023] P3 sofreu preempção -> fila BAIXA
        CPU executa P2 por 1 unidades
[t=024] P2 finalizado
        CPU executa P3 por 2 unidades
[t=026] P3 finalizado
=====

```

Interpretação dos eventos principais:

- t=0: P0 é admitido na fila alta e imediatamente selecionado. Executa por apenas 2 unidades porque seu I/O de DISCO está agendado para `inicioProximoIo=2`.

- t=2: P0 solicita I/O de DISCO e vai para a fila de I/O. A CPU fica aguardando o próximo processo pronto.
- t=4: P1 chega e entra na fila alta. Executa por 3 unidades (um quantum completo). Como não finalizou e não solicitou I/O, sofre preempção e vai para a fila baixa.
- t=10: P0 retorna do DISCO (bloqueado de t=2 a t=10, 8 u.t. de I/O). Vai para a fila baixa. Executa as 2 unidades restantes e finaliza.
- Os processos P2 e P3 (sem I/O) serão preemptados repetidamente entre as filas baixa e baixa até completar seu tempo de CPU.

### 7.3 Resumo Final

Ao término da simulação, o método imprimirResumoFinal() exibe uma tabela com métricas por processo e totais globais:

```
=====
RESUMO FINAL
=====
```

| PID | Chegada | Fim | Temp Espera | Turnaround | Preemp. |
|-----|---------|-----|-------------|------------|---------|
| P0  | 0       | 20  | 8           | 20         | 0       |
| P1  | 4       | 9   | 0           | 5          | 1       |
| P2  | 8       | 24  | 9           | 16         | 2       |
| P3  | 12      | 26  | 6           | 14         | 2       |

```
-----
Tempo total da simulação : 26 unidades
CPU ociosa                : 2 unidades
Total de preempção        : 5
Tempo de espera médio     : 5,75
Turnaround médio          : 13,75
=====
```

O turnaround de cada processo é calculado como a diferença entre o instante de finalização e o instante de chegada ( $\text{instanteFinalizacao}[i] - \text{tempoChegada}[i]$ ). O tempo de espera acumula os períodos em que o processo estava em estado PRONTO mas não estava na CPU. A CPU ociosa representa o número de unidades de tempo em que nenhum processo estava disponível para execução.

## 8 CONCLUSÃO

O desenvolvimento deste simulador de escalonamento de processos proporcionou ao grupo uma compreensão aprofundada dos mecanismos internos de um sistema operacional, em particular no que diz respeito ao gerenciamento e à priorização de processos. A implementação do algoritmo Round Robin com Feedback em Java permitiu traduzir conceitos teóricos em código funcional, reforçando a aprendizagem prática da disciplina de Sistemas Operacionais.

Entre os aprendizados mais relevantes, destaca-se a compreensão do impacto do quantum na eficiência do escalonamento: valores muito pequenos aumentam o overhead de trocas de contexto, enquanto valores grandes podem prejudicar a responsividade de processos interativos. A distinção entre processos CPU-bound e I/O-bound, materializada nas duas filas de prioridade do simulador, evidenciou como os sistemas operacionais modernos buscam equilibrar a utilização de recursos com a experiência do usuário.

A modelagem do PCB por meio de arrays paralelos, embora funcional para o escopo do trabalho, evidenciou uma limitação de legibilidade e manutenibilidade em relação a uma abordagem orientada a objetos com uma classe Processo encapsulando todos os atributos. Da mesma forma, a ausência de geração dinâmica e aleatória de processos limita a capacidade de testar o simulador em cenários variados, sendo este um ponto de melhoria identificado pelo próprio grupo no README do projeto.

Como melhorias futuras, o grupo propõe:

- Encapsular o PCB em uma classe Java (Processo ou ProcessControlBlock), melhorando a organização e a coesão do código.

- Implementar a lógica de PPID, permitindo representar relações de hierarquia entre processos e aproximando a simulação do funcionamento de sistemas operacionais reais, onde processos podem criar processos filhos.
- Adicionar suporte a múltiplos dispositivos de I/O simultâneos, onde a fila de I/O atual processa apenas um dispositivo por vez de forma sequencial.
- Implementar uma terceira fila de prioridade ou adotar aging (envelhecimento), evitando que processos na fila baixa fiquem em starvation quando há grande fluxo de processos novos.
- Desenvolver uma interface gráfica ou saída visual tipo diagrama de Gantt para facilitar a visualização do escalonamento.

Em síntese, o trabalho cumpriu seu objetivo pedagógico de simular e demonstrar o funcionamento do algoritmo Round Robin com Feedback, contribuindo significativamente para a formação do grupo em conceitos fundamentais de sistemas operacionais.

## **REFERÊNCIAS**

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson Education do Brasil, 2016.

DEITEL, Harvey M.; DEITEL, Paul J. Java: Como Programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.